omniBox

Multi-Agent Autonomy System

Groq LLaMA 4 | Claude Opus | GitHub API | Anvil MMXXVI

Delegate **any** natural-language goal. An orchestrator decomposes it into a task graph, assigns specialist AI agents, invokes real tools, and delivers verified results — fully autonomously.

Business Use Case

Software teams lose **hours every day** routing bug reports, writing fix branches, opening PRs, updating CI pipelines, and chasing code reviews. omniBox eliminates that toil entirely.

Scenario	Without omniBox	With omniBox
GitHub issue filed	Dev reads, reproduces, fixes, opens PR — 2–4 h	Agents fix & PR in < 5 min, zero human input
Add CI/CD pipeline	DevOps writes YAML, commits, iterates — 1–2 h	One sentence → working GitHub Actions workflow + PR
Architecture review	Senior engineer spends half a day documenting	Researcher reads repo → Writer produces Mermaid diagram + report
Codebase research	Hours of grep, reading, note-taking	Researcher agent traverses & summarises in seconds

Target Market

- Engineering teams shipping ≥ 5 GitHub repos who want automated issue-to-PR pipelines
- Platform / DevOps squads maintaining CI/CD configurations across many repos
- CTOs and tech leads who need instant codebase architecture reports
- Startups with small teams who can't afford senior DevOps headcount
- Open-source maintainers drowning in bug reports and contributor PRs

Value Proposition

omniBox is **provider-agnostic** (40 models across Groq, Anthropic, OpenAI, Google, Mistral), **restart-resumable** (SQLite WAL + lease reclaim), **self-healing** (auto-files issues and spawns fix goals on developer errors), and **fully observable** (Omium SDK tracing, live SSE dashboard). No proprietary lock-in. Deploy anywhere.

Agent Architecture

omniBox is built on a **generic multi-agent execution engine**. The orchestrator (Claude) plans any goal as a directed acyclic graph (DAG) of tasks. Each task is assigned to a specialist agent that runs a tool-call loop until it calls `submit_result`.

Orchestrator (Claude / Groq LLaMA 4 Maverick)

Receives the natural-language goal → emits a **PlanSchema**: a list of TaskSpecs with IDs, agents, inputs, and dependency edges. Uses forced tool-call (`submit_plan`) with 5-attempt retry + Groq *failed_generation* salvage. Task IDs are prefixed with the goal UUID to prevent UNIQUE constraint collisions across concurrent goals.

Specialist Agents

Agent	Model	Tools	Output Schema
researcher	Groq LLaMA 4 Maverick	web_search · http_request · github_read_file · github_list_dir · github_get_issue · github_search_code · spawn_goal	`{summary, key_points, sources, code_context}`
writer	Groq LLaMA 3.3 70B	file_ops	`{text, title}`
coder	Groq LLaMA 3.3 70B	code_exec · file_ops · web_search · github_read_file	`{code, output, success}`
integrator	Groq LLaMA 3.3 70B	github_pr · github_post_comment · github_create_repo · github_list_workflows · github_set_branch_protection · http_request · wait_webhook · spawn_goal	`{action, result, url}`

Autonomy & Resilience Features

- **Dynamic Replanning** — When a task hits max retries, the orchestrator is called again with context of completed tasks + failure reason. It devises an alternative sub-plan and inserts new tasks — the goal continues instead of dying.
- **spawn_goal tool** — Any agent can call spawn_goal mid-execution to autonomously create a new goal. Used when the researcher discovers a complex fix that requires the full researcher → coder → integrator pipeline.
- **Retry with failure context** — Each retry injects the previous error into the agent's user message, forcing a different approach. Consecutive-error counter nudges the agent to submit with partial results after 3 consecutive failures.
- **Self-heal loop** — error_classifier.py detects developer-side bugs vs. external errors. On a developer bug: auto-files a GitHub issue on the omniBox repo, then spawns a fix goal (researcher → coder → integrator) to patch and PR the root cause.
- **Tool idempotency** — Every tool call is hashed (task_id + tool + args). Re-runs return the stored result — no duplicate Slack posts, no double PRs, crash-safe.
- **Lease reclaim** — Worker reclaims RUNNING tasks with expired leases every 30 s. Restart-resumable from any failure point.

LLM Layer & Fallback Chain

LiteLLM wraps all providers. Per-model fallback chains handle hard quota errors (daily limit, resource_exhausted) by trying the next candidate, and soft rate limits by sleeping the declared retry delay. Claude 4 models exclude the `temperature` parameter (API constraint). 40 models pre-configured; any LiteLLM-compatible string also accepted at runtime.

Overall Workflow

omniBox follows a **plan** → **execute** → **observe** loop backed by SQLite WAL for full restart-resumability. Three asyncio workers run concurrently inside the FastAPI process — no separate queue or Redis needed.

Step	Component	Detail
1	**Goal Submission**	POST /api/goals — inserts row with status=NEW, generates trace_id for Omium
2	**Orchestration**	goal_planner_loop claims NEW goal → calls Claude → emits PlanSchema DAG → inserts task rows (deps=[] → READY, others → PENDING)
3	**Task Execution**	task_executor_loop (Semaphore 5) claims READY tasks → resolves {{t1.output.field}} interpolations → runs agent_runner.run()
4	**Agent Loop**	acompletion() loop: builds messages → calls LLM → for each tool_call checks idempotency → invokes tool → emits SSE events → repeats until submit_result
5	**Dependency Promotion**	On DONE: promote_ready_tasks() SQL query unblocks tasks whose all deps are DONE → they become READY for the executor
6	**Goal Completion**	When terminal task completes → goal.status=COMPLETED, output stored, goal_done SSE event fires → frontend renders output
7	**Failure Recovery**	On max_attempts exceeded → try dynamic replan (one chance). On developer error → self-heal: file issue + spawn fix goal. On crash → reclaim_loop restores RUNNING tasks to READY

Demo Flow — GitHub Issue → Auto-PR

GitHub issue opened	Webhook fires → POST /api/webhooks/github → goal created automatically
Orchestrator plans	researcher → coder → integrator DAG emitted in < 2 s
Researcher agent	github_list_dir explores repo, github_read_file reads relevant files, github_get_issue fetches bug details
Coder agent	Writes Python fix, executes via code_exec sandbox, captures real output
Integrator agent	github_pr commits fixed files, opens cross-repo PR with professional body, github_post_comment posts PR link on original issue
Live dashboard	Every tool call, message, and status transition streams via SSE to the React frontend in real time

Technology Stack

Backend — FastAPI · SQLite WAL · aiosqlite · LiteLLM · Pydantic v2

LLMs — Groq (LLaMA 4 Maverick / LLaMA 3.3 70B) · Anthropic Claude · 40 models total

Tools — Tavily search · PyGithub · httpx · asyncio subprocess (code_exec) · SSE (events.py)

Frontend — React 18 · TypeScript · Vite · Tailwind CSS · Framer Motion · React Flow · Mermaid.js

Tracing — Omium SDK — goal_trace_context → orchestrator span → task spans → tool spans

Auth — Firebase Auth (email + Google + GitHub OAuth)

GitHub: https://github.com/viscous106/omniBox · Built for Anvil MMXXVI — Multi-Agent Autonomy Track